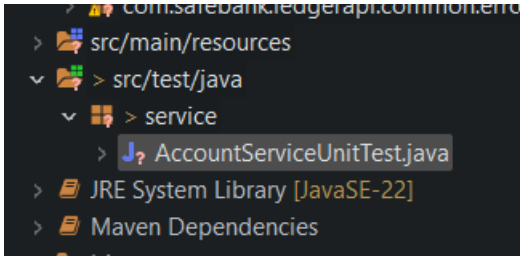
SpringBoot Service tests:

Unit tests

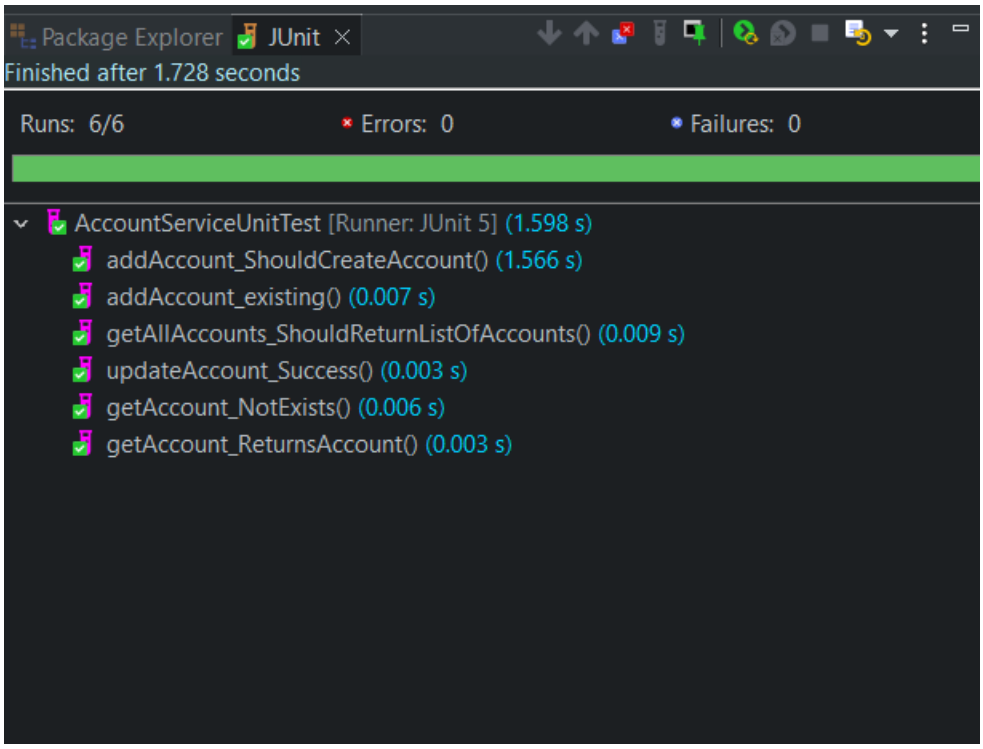
This is same as Java “Junit” tests

Go to src/test/java => create a package “service”



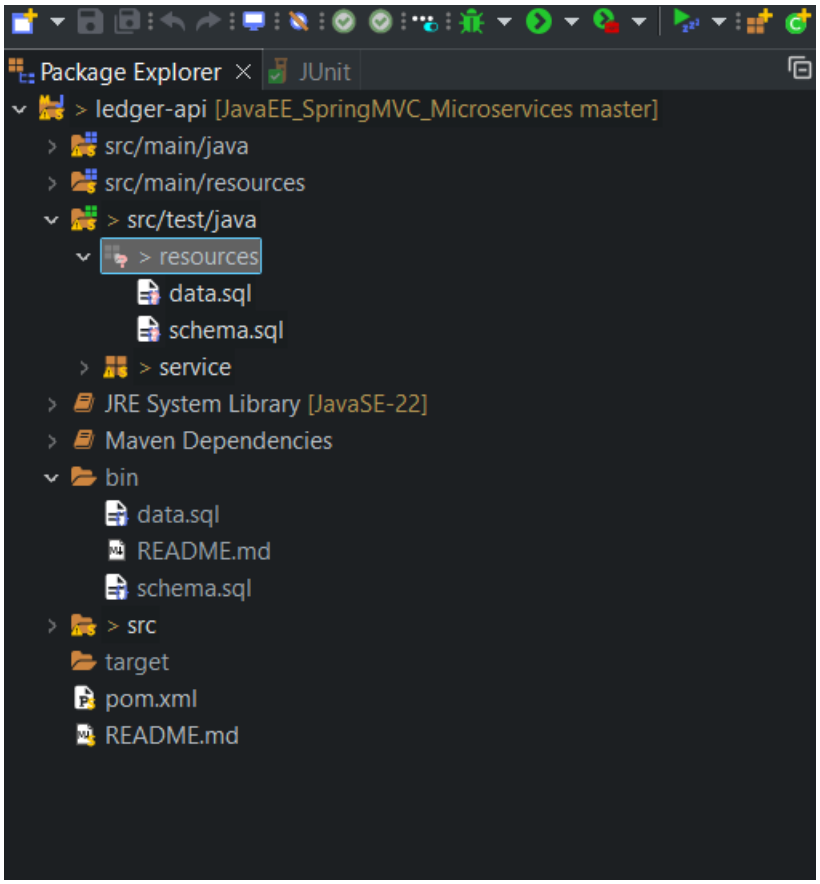
Create a file AccountServiceUnitTest.java

Right-click the Test file and run Junit



Integration tests:

Setup => go to src/test/java => create a new package “resources”. Copy the .sql files into this folder from /bin



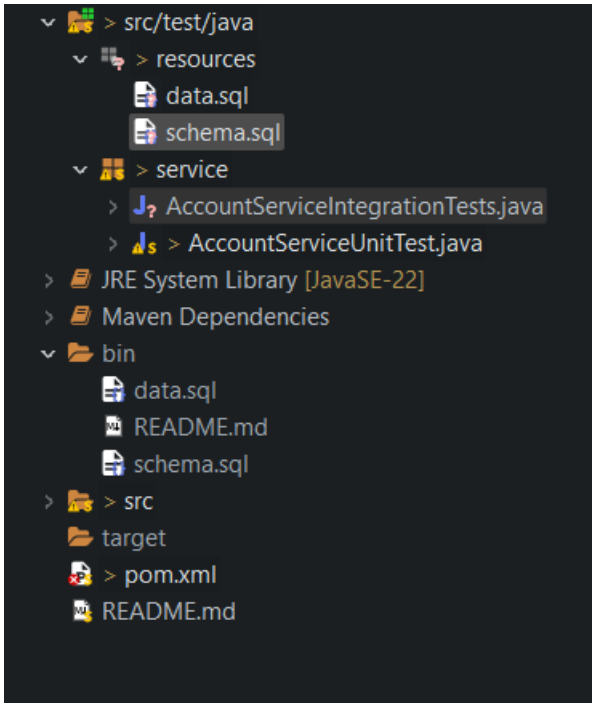
Go to pom.xml and add a new dependency => note “scope” is “test”

```
<dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>test</scope>
</dependency>
```

```
<dependencies>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
```

Re-run the application

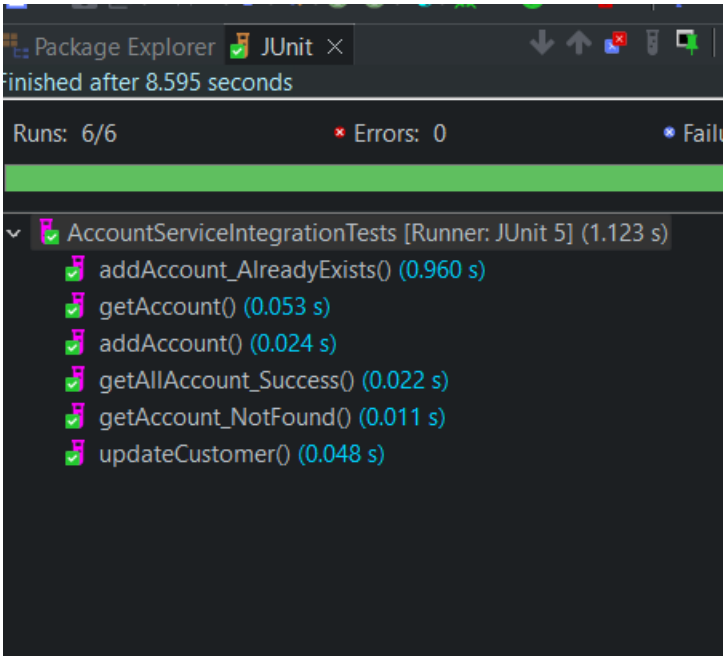
Create a new Java class => AccountServiceIntegrationTests.java



required for SpringBoot tests
// instead of mocking, we will use the Embedded database H2, and we are running the actual seed .sql scripts on H2
@SpringBootTest(classes = com.safebank.ledgerapi.LedgerApiApplication.class)
@ActiveProfiles("test")
//@AutoConfigureTestDatabase(replace=Replace.ANY)
public class AccountServiceIntegrationTests {

# Force Spring Boot to run your schema.sql and data.sql scripts
spring.sql.init.mode=always

Tell Hibernate to let your manual schema.sql script build the tables
spring.jpa.hibernate.ddl-auto=none



Web Integration tests:

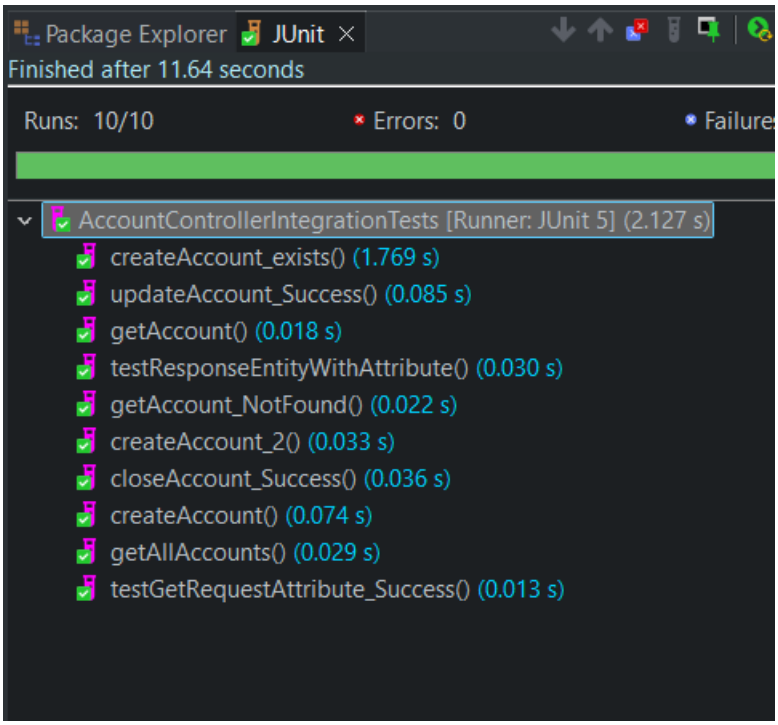
under ledger-api/src/test/java , create new package: “web.controllers”

MockMvc mockMVC; // creates a pseudo-dispatcher servlet, which allows us to test the method as though it were running in its own servlet container

```
@GetMapping("/profile")
public String getProfileContext(@RequestAttribute("userEmail") String email) {
    // Business logic running against the secure request attribute
    return "Secure session active for account user: " + email;
}


@GetMapping("/profile1")
public ResponseEntity<String> getProfileWithEntity(HttpServletRequest request) {
    // 1. Pull the attribute out of the incoming request
    String email = (String) request.getAttribute("userEmail");

    // 2. Wrap it inside a standard ResponseEntity container
    return ResponseEntity.ok()
            .header("X-Account-Context", "Verified")
            .body("Active Account Profile for: " + email);
}
```



Look into Environment and Profiles: mock Environment and test before deploying
Evaluate how to mock clients using the mock RestTemplate construct
Build aggregate objects from database tables using joins
Write some requirements on what that methods should return, first
Write the code to make your tests pass (TDD)

Cyclomatic complexity is a software metric used to measure the structural complexity of a program. It counts the number of linearly independent paths through your source code.
Evaluate Cyclomatic complexity

Build a simple web UI and test that